GEN -AI HANDS ON UNIT 2

NAME:ANNSHUL SHETTY

SRN:PES2UG23CS079

SEM:6 SEC:B

[3]
26s

```
from langchain_google_genai import ChatGoogleGenerativeAI

# Model A: The "Accountant" (Precision)
llm_focused = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.0)

# Model B: The "Poet" (Creativity)
llm_creative = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=1.0)
```

7. Experiment: Consistency vs. Creativity

We will ask both models to "Define the word 'Idea' in one sentence." We will run the code **TWICE** for each model.

Hypothesis:

- The Focused model (Temp=0) should say the *exact same thing* both times.
- The Creative model (Temp=1) should say *different things*.

[4]
11s

```
prompt = "Define the word 'Idea' in one sentence."

print("--- FOCUSED (Temp=0) ---")
print(f"Run 1: {llm_focused.invoke(prompt).content}")
print(f"Run 2: {llm_focused.invoke(prompt).content}")
```

```
--- FOCUSED (Temp=0) ---
Run 1: An idea is a thought, concept, or mental image formed in the mind.
Run 2: An idea is a thought, concept, or suggestion that is formed or exists in the mind.
```

[5]
10s

```
print("--- CREATIVE (Temp=1) ---")
print(f"Run 1: {llm_creative.invoke(prompt).content}")
print(f"Run 2: {llm_creative.invoke(prompt).content}")
```

```
--- CREATIVE (Temp=1) ---
Run 1: An idea is a thought, concept, plan, or mental impression formed in the mind.
Run 2: An idea is a concept, thought, or mental impression formed in the mind.
```

Inference: LangChain provides a standardized class (ChatGoogleGenerativeAI) to configure "model brains." The most critical parameter is temperature, which dictates the balance between precision (0.0) and randomness (1.0).

3. Method B: The LCEL Way (Good)

We use the **Pipe Operator (|)**. It works just like Unix pipes: pass the output of the left side to the input of the right side.

[13]
2s

```
# Define the chain once
chain = template | llm | parser

# Invoke the whole chain
print(chain.invoke({"topic": "Octopuses"}))
```

```
Octopuses have **three hearts**! Two pump blood through their gills, and a third circulates blood to the rest of their body.

And because of a copper-rich protein called hemocyanin (instead of iron-rich hemoglobin like ours), their blood is **blue**!
```

Inference: The | operator allows for a declarative way to compose chains. Instead of nested function calls, the flow from Template to Model to Parser is linear and human-readable,

facilitating easy component swapping.

3. Hallucination vs. Creativity

Did the model make up a reason? Since we didn't give it facts, it **Predicted the most likely reason** (Usually "Experience" or "Volume of applications").

This is NOT a bug. It is a feature. The model is *completing the pattern* of a rejection email.

```
structured_prompt = """
# Context
You are an HR Manager at a quirky startup called 'RocketBoots'.

# Objective
Write a rejection email to a candidate named Bob.

# Constraints
1. Be extremely brief (under 50 words).
2. Do NOT say 'we found someone better'. Say 'the role changed'.
3. Sign off with 'Keep flying'.

# Output Format
Plain text, no subject line.
"""

print("--- STRUCTURED PROMPT ---")
print(llm.invoke(structured_prompt).content)
```

```
--- STRUCTURED PROMPT ---
Hi Bob,

Thank you for your interest in RocketBoots. We appreciate your time and effort.

While your application was impressive, the requirements for this role have recently changed. We won't be moving forward with your candidacy at this time.

Keep flying,
RocketBoots HR
```

Inference: Using Markdown headers (# Context, # Objective) within prompt strings helps the model delineate between background information and specific constraints, leading to higher adherence to word counts and specific phrases.

2. Dynamic Few-Shotting

If you have 1000 examples, you can't fit them **all** in the context window. We use a **Selector** to pick the best ones.

The Selector Flow (Flowchart)

```
graph LR
    Input[User Input] -->|Semantic Search| Database[Example Database]
    Database -->|Top 3 Matches| Selector
    Selector -->|Inject| Prompt
```

```
from langchain_core.prompts import ChatPromptTemplate, FewShotChatMessagePromptTemplate

# 1. Our Database of Examples
examples = [
    {"input": "The internet is down.", "output": "We are observing connectivity latency."},
    {"input": "This code implies a bug.", "output": "The logic suggests unintended behavior."},
    {"input": "I hate this feature.", "output": "This feature does not align with my preferences."},
]

# 2. Template for ONE example
example_fmt = ChatPromptTemplate.from_messages([
    ("human", "{input}"),
    ("ai", "{output}")
])

# 3. The Few-Shot Container
few_shot_prompt = FewShotChatMessagePromptTemplate(
    example_prompt=example_fmt,
    examples=examples
)

# 4. The Final Chain
final_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a corpo-speak Translator. Rewrite the input to sound professional."),
    few_shot_prompt,      # Inject examples here
    ("human", "{text}")
])

chain = final_prompt | llm

print(chain.invoke({"text": "This app sucks."}).content)
```

```
The application's current state requires optimization.
```

Inference: FewShotChatMessagePromptTemplate automates the formatting of examples into consistent Human/AI message pairs. This is more robust than manual string concatenation for

maintaining the "roles" understood by modern chat models.

3. The Experiment: A Tricky Math Problem

Let's try a problem that requires multi-step logic.

Problem: "Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many does he have now?"

```
[ ]
question = "Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many does he have now?"

# 1. Standard Prompt (Direct Answer)
prompt_standard = f"Answer this question: {question}"
print("--- STANDARD (Llama3.1-8b) ---")
print(llm.invoke(prompt_standard).content)
```

```
--- STANDARD (Llama3.1-8b) ---
To find out how many tennis balls Roger has now, we need to add the initial number of tennis balls he had (5) to the number of tennis balls he bought (2 cans * 3 tennis balls per can).

2 cans * 3 tennis balls per can = 6 tennis balls

Now, let's add the initial number of tennis balls (5) to the number of tennis balls he bought (6):

5 + 6 = 11

So, Roger now has 11 tennis balls.
```

Inference: Adding "Let's think step by step" to a prompt forces the model to attend to its own intermediate reasoning tokens. This is the difference between a model making a calculation error and the model "self-correcting" through verbalization.

We will generate 3 distinct solutions for a problem and then use a "Judge" to pick the best one.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableParallel, RunnableLambda
from langchain_core.output_parsers import StrOutputParser

problem = "How can I get my 5-year-old to eat vegetables?"

# Step 1: The Branch Generator
prompt_branch = ChatPromptTemplate.from_template(
    "Problem: {problem}. Give me one unique, creative solution. solution {id}:"
)

branches = RunnableParallel(
    sol1=prompt_branch.partial(id="1") | llm | StrOutputParser(),
    sol2=prompt_branch.partial(id="2") | llm | StrOutputParser(),
    sol3=prompt_branch.partial(id="3") | llm | StrOutputParser(),
)

# Step 2: The Judge
prompt_judge = ChatPromptTemplate.from_template(
    """
    I have three proposed solutions for: '{problem}'

    1: {sol1}
    2: {sol2}
    3: {sol3}

    Act as a Child Psychologist. Pick the most sustainable one (not bribery) and explain why.
    """
)

# Chain: Input -> Branches -> Judge -> Output
tot_chain = (
    RunnableParallel(problem=RunnableLambda(lambda x: x), branches=branches)
    | (lambda x: {**x["branches"], "problem": x["problem"]})
    | prompt_judge
    | llm
    | StrOutputParser()
)

print("--- Tree of Thoughts (ToT) Result ---")
print(tot_chain.invoke(problem))
```

```
--- Tree of Thoughts (ToT) Result ---
As a child psychologist, I would recommend **Solution 3: The "Vegetable Pizza Party"** as the most sustainable approach to encourage your 5-year-old to eat vegetables. Here's why:

1. **Empowerment**: By letting your child be the chef, you're giving them a sense of control and agency over their food choices. This empowerment can foster a positive relationship with vegetables and mealtime in general.
2. **Intrinsic motivation**: When children are involved in the cooking process, they're more likely to be motivated to try new foods because they've invested their own effort and creativity. This intrinsic motivation is more sustainable than relying on external rewards or bribes.
3. **Learning through exploration**: The "Vegetable Pizza Party" approach encourages your child to explore and learn about different vegetables in a fun and engaging way. By choosing colors and patterns, they're developing their fine motor skills, hand-eye coordination, and creativity.
4. **Transferable skills**: The skills your child develops through this activity, such as measuring, mixing, and arranging, can be transferred to other areas of their life, like math, science, and social skills.
5. **Long-term effects**: By making mealtime a positive and enjoyable experience, you're laying the groundwork for a lifelong healthy relationship with food. This approach can help your child develop healthy eating habits and a willingness to try new foods, which can benefit them throughout their lives.

In contrast, while **Solution 1: Create a "Veggie Face"** and **Solution 2: Sneaky Ways to Add Vegetables** may be effective in the short term, they may not be as sustainable in the long term. Creating a "Veggie Face" may be seen as a gimmick, and children may eventually lose interest. Sneaky ways to add v

Overall, the "Vegetable Pizza Party" approach offers a holistic and empowering approach to encouraging your child to eat vegetables, making it the most sustainable solution.
```

Inference: The implementation uses RunnableParallel to generate disparate ideas (Sci-Fi, Romance, Horror) which are then "converged" by a final prompt. This code pattern mimics a collaborative brainstorming session.

3. The Math: Cosine Similarity

How do we know if two vectors are close? We measure the **Angle** between them.

Cosine Similarity Formula

$$\text{Similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|}$$

- **1.0**: Arrows point in the Exact Same Direction (Identical).
- **0.0**: Arrows are Perpendicular (Unrelated).
- **-1.0**: Arrows point in Opposite Directions (Opposite).

Experiment: Let's compare "Cat", "Dog", and "Car".

```python
import numpy as np

def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

vec_cat = embeddings.embed_query("Cat")
vec_dog = embeddings.embed_query("Dog")
vec_car = embeddings.embed_query("Car")

print(f"Cat vs Dog: {cosine_similarity(vec_cat, vec_dog):.4f}")
print(f"Cat vs Car: {cosine_similarity(vec_cat, vec_car):.4f}")
```

```
Cat vs Dog: 0.6606
Cat vs Car: 0.4633
```

Inference: Text is converted into a list of floating-point numbers (embeddings) so that semantic similarity can be calculated via high-dimensional math (Cosine Similarity) rather than simple keyword matching.

4. The RAG Chain

We use LCEL to stitch it together.

Step 1: The `retriever` takes the question, converts it to numbers, and finds the closest document. **Step 2:** `RunnablePassthrough` holds the question. **Step 3:** The `prompt` combines them.

```python
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

template = """
Answer based ONLY on the context below:
{context}

Question: {question}
"""
prompt = ChatPromptTemplate.from_template(template)

chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)

result = chain.invoke("What is the secret password?")
print(result)
```

```
The secret password to the lab is 'Blueberry'.
```

Inference: The RAG chain logic maps the user query to a dictionary where one key is dynamically populated by the retriever. This "injects" external facts into the prompt before it ever reaches the LLM.